



UNIVERSITÀ DEGLI STUDI
DI SALERNO

Relazione Tecnica di Progettazione e Sviluppo

Study Planner & Exam Tracker

Corso di Mobile Programming

Gruppo 21

Team di Sviluppo:

Gerardo Carino — Matricola: 0612710164

Donato Finiello — Matricola: 0612709377

Mara Mariano — Matricola: 0612709247

Cataldo Sabato — Matricola: 0612709472

Daniel Vita — Matricola: 0612709007

Anno Accademico 2025/2026

Indice

1	Descrizione dell'app	2
1.1	Obiettivo dell'applicazione	2
1.2	Tipologia di utenti a cui è rivolta	2
1.3	Problema che l'app intende risolvere	2
1.4	Principali scenari d'uso	3
2	Requisiti	4
2.1	Specifica dei Requisiti	4
2.1.1	Requisiti Funzionali (RF)	4
2.1.2	Requisiti Non Funzionali (RNF)	6
2.2	Feature avanzate scelte	7
2.3	Funzionalità considerate ma non implementate	8
2.4	Limitazioni note	8
3	Progettazione dell'app	9
3.1	Struttura generale dell'app	9
3.2	Schermate principali	9
3.3	Flusso di navigazione	9
3.4	Organizzazione dei dati	10
3.5	Wireframe	10
4	Scelte tecnologiche	12
4.1	Framework utilizzato e SDK	12
4.2	Librerie principali e Architettura UI	12
4.3	Gestione avanzata dei Modali e della Tastiera Nativa	12
4.4	Ottimizzazione del Rendering: Approccio Ibrido FlatList vs map()	13
4.5	Modalità di gestione della persistenza	13
5	Implementazione	14
5.1	Organizzazione del codice e Alberatura del Progetto	14
5.2	Componenti e Widget Principali	15
5.3	Gestione dello Stato Globale e Persistenza dei Dati	15
5.4	Parti particolarmente significative o complesse	16
6	Manuale Operativo	17
6.1	Prerequisiti di sistema	17
6.2	Istruzioni per l'installazione e l'esecuzione	17
7	Metodologia di Lavoro (Extra)	18
7.1	Organizzazione del team e versionamento	18
8	Conclusioni e Sviluppi Futuri (Extra)	19
8.1	Considerazioni finali	19
8.2	Roadmap per la Versione 2.0	19

1 Descrizione dell'app

StudyPlanner Pro è un'applicazione mobile cross-platform concepita per la gestione della vita accademica e l'ottimizzazione dei flussi di studio.

1.1 Obiettivo dell'applicazione

L'obiettivo primario dell'applicazione è gestire la carriera accademica dello studente (monitoraggio dei corsi, pianificazione degli appelli d'esame, calcolo metrico dei CFU e della media ponderata) dando risalto alla produttività quotidiana (gestione dei task, agenda giornaliera e time-tracking tramite tecnica Pomodoro).

A differenza dei comuni strumenti di tracciamento, StudyPlanner Pro non si limita a memorizzare i dati, ma li correla attivamente: il tempo registrato focalizzandosi su un singolo task incrementa dinamicamente le statistiche globali di preparazione del corso associato.

1.2 Tipologia di utenti a cui è rivolta

Il target di riferimento è composto interamente da studenti universitari (iscritti a corsi di laurea triennale, magistrale o a ciclo unico) che avvertono l'esigenza di una pianificazione rigorosa delle proprie attività. L'applicazione si rivolge in particolare a:

- **Studenti che faticano a gestire le scadenze**, grazie a un sistema di alert visivi e un assistente predittivo.
- **Studenti orientati ai dati (data-driven)**, che desiderano monitorare quantitativamente le ore di focus per ottimizzare il proprio rendimento e bilanciare lo sforzo tra le diverse discipline.
- **Utilizzatori di metodologie di time-management**, che cercano uno strumento integrato per applicare la tecnica Pomodoro senza dipendere da applicazioni terze e frammentate.

1.3 Problema che l'app intende risolvere

Nel contesto universitario attuale, lo studente si trova a dover fronteggiare alcune problematiche principali:

- **Frammentazione degli strumenti digitali:** Spesso gli studenti sono costretti a utilizzare più applicazioni contemporaneamente (il calendario nativo dello smartphone per gli appelli, fogli di calcolo Excel per la media ponderata, app esterne per la checklist dei task e timer fisici per il focus). Questo causa dispersione di informazioni.
- **Mancanza di metriche sul carico di lavoro (Overhead cognitivo):** È difficile per uno studente quantificare quanto tempo stia effettivamente dedicando a una materia rispetto a un'altra, col rischio di arrivare a ridosso di una scadenza con una preparazione insufficiente.

- **Procrastinazione e paralisi decisionale:** Davanti a un elevato numero di esami e consegne, lo studente sperimenta difficoltà nello stabilire le priorità giornaliere.

StudyPlanner Pro risolve alla radice queste criticità accentrando i dati e offrendo un Assistente di Studio che suggerisce l'azione prioritaria da compiere in base alla prossimità temporale degli esami.

1.4 Principali scenari d'uso

Per comprendere appieno l'utilità dell'applicativo, si delineano di seguito i flussi di interazione tipici (Use Cases) da parte dell'utente finale:

- **Pianificazione e Focus Giornaliero:** L'utente apre l'applicazione all'inizio della giornata, consulta l'agenda per visualizzare i task programmati e avvia il timer Pomodoro integrato. Al termine della sessione, il sistema marca il task come completato e aggiorna automaticamente il monte ore dedicato a quella specifica materia.
- **Integrazione di un nuovo Insegnamento:** All'inizio di un semestre, l'utente inserisce i nuovi corsi definendo i CFU e la data presunta dell'appello d'esame. Il sistema genera un cruscotto dedicato dove l'utente può iniziare a frammentare lo studio in micro-obiettivi.
- **Monitoraggio della Carriera e Intervento dell'Assistente:** A ridosso della sessione d'esame, l'utente consulta la sezione analitica per verificare l'impatto di un eventuale voto sulla media ponderata. Qualora vi siano attività in ritardo, l'Assistente di Studio riorganizza dinamicamente le priorità suggerendo all'utente su quale materia concentrarsi per colmare le lacune.

2 Requisiti

2.1 Specifica dei Requisiti

L'analisi dei requisiti definisce in modo astratto e atomico le capacità che l'applicativo deve possedere (funzionali) e i vincoli qualitativi, operativi e tecnologici a cui deve sottostare (non funzionali), senza anticipare alcuna scelta tecnologica o di implementazione software.

2.1.1 Requisiti Funzionali (RF)

I requisiti funzionali descrivono i servizi che il sistema deve offrire in risposta agli input dell'utente e il comportamento dell'applicativo in scenari specifici.

Modulo Gestione Carriera e Insegnamenti

ID	Titolo	Descrizione
RF-1.1	Registrazione Corso	Il sistema deve consentire all'utente di inserire un nuovo corso accademico indicandone il nome, il docente, il peso in crediti formativi (CFU) e un voto obiettivo (Target).
RF-1.2	Stato Iniziale Insegnamento	All'atto della creazione di un corso, il sistema deve permettere all'utente di specificare lo stato di avanzamento iniziale, selezionando unicamente tra "In Corso" o "Da Iniziare".
RF-1.3	Consultazione Catalogo Corsi	Il sistema deve mostrare un riepilogo globale di tutti i corsi inseriti, offrendo la possibilità di filtrarli in base allo stato di avanzamento.
RF-1.4	Ricerca Testuale	Il sistema deve permettere la ricerca rapida dei corsi tramite l'inserimento di parole chiave corrispondenti al nome del corso o al cognome del docente.
RF-1.5	Modifica e Rimozione	Il sistema deve consentire la modifica dei metadati di un corso esistente e la sua eliminazione definitiva dal profilo utente.
RF-1.6	Avanzamento di Stato	Il sistema deve aggiornare in modo automatico o manuale lo stato di un insegnamento (es. da "Da Iniziare" a "In Corso").

Modulo Esami e Verbalizzazione

ID	Titolo	Descrizione
RF-2.1	Pianificazione Appello	Il sistema deve permettere di fissare una data di esame o una scadenza accademica, memorizzandone il titolo e la tipologia di prova.
RF-2.2	Vincolo di Associazione Esame	Il sistema deve consentire l'associazione di una data d'esame esclusivamente a un corso il cui stato sia "Da Iniziare" o "In Corso". Il sistema deve inibire la pianificazione per insegnamenti già contrassegnati come superati.
RF-2.3	Categorizzazione Prove	Il sistema deve supportare la classificazione delle prove d'esame in categorie distinte (Scritto, Orale, Progetto).
RF-2.4	Verbalizzazione dell'Esito	Il sistema deve consentire la registrazione dell'esito di un esame pianificato. L'utente deve poter marcare la prova esclusivamente come "Superato" (con immissione del voto) o "Respinto".
RF-2.5	Chiusura Carriera del Corso	Al momento della registrazione di un esame come "Superato", il sistema deve convertire automaticamente lo stato del corso associato in "Superato", inserendo il voto finale nel libretto dello studente.
RF-2.6	Cancellazione Scadenza	Il sistema deve consentire la revoca o l'eliminazione di un appello precedentemente programmato.

Modulo Task, Agenda e Time-Tracking

ID	Titolo	Descrizione
RF-3.1	Creazione Attività (Task)	Il sistema deve permettere la scomposizione dello studio in micro-attività, definendo per ciascuna un titolo, un livello di priorità e un tempo stimato di esecuzione.
RF-3.2	Classificazione del Task	Il sistema deve consentire di legare un task a un corso specifico oppure di marcarlo come “Generale” per attività non didattiche.
RF-3.3	Checklist Interattiva	Il sistema deve offrire una vista giornaliera in cui l’utente può marcare istantaneamente i task come completati, modificando lo stato visivo dell’attività.
RF-3.4	Navigazione Temporale	Il sistema deve presentare un’interfaccia di pianificazione settimanale per consultare e inserire i task nei diversi giorni della settimana.
RF-3.5	Timer di Focus	Il sistema deve integrare un modulo per la gestione di intervalli di tempo dedicati allo studio intensivo alternati a periodi di riposo (Tecnica Pomodoro).
RF-3.6	Associazione Timer-Task	Durante l’attivazione del timer, il sistema deve permettere all’utente di selezionare quale task o corso si intende approfondire.
RF-3.7	Log delle Sessioni	Al completamento di un ciclo di focus, il sistema deve registrare in modo silente l’attività nell’archivio storico, memorizzando data, durata e corso di riferimento.

Modulo Dashboard e Consolle Analitica

ID	Titolo	Descrizione
RF-4.1	Indicatori di Carriera	Il sistema deve calcolare e mostrare in tempo reale la media ponderata dei voti conseguiti e il conteggio cumulativo dei CFU acquisiti rispetto al totale previsto.
RF-4.2	Cruscotto di Controllo (Home)	La schermata principale deve aggregare i dati mostrando i contatori delle attività odierne, lo stato della carriera e gli eventi più vicini nel tempo.
RF-4.3	Analisi del Tempo Speso	Il sistema deve visualizzare graficamente la distribuzione del tempo totale di studio effettivo, ripartendolo tra le diverse materie in base ai log del timer.
RF-4.4	Confronto Avanzamento	Il sistema deve mostrare il livello di completamento dei task associati a ciascun corso per monitorare lo stato della preparazione rispetto al voto target impostato.

2.1.2 Requisiti Non Funzionali (RNF)

I requisiti non funzionali stabiliscono i vincoli di qualità, sicurezza, usabilità e le restrizioni operative del sistema.

ID	Categoria	Descrizione del Vincolo
RNF-1.1	Usabilità (UX)	L'interfaccia utente deve essere ottimizzata per l'uso su dispositivi mobili, garantendo che i form di inserimento dati non vengano mai occultati dagli elementi della tastiera di sistema.
RNF-1.2	Usabilità (UX)	La tastiera di inserimento dati deve poter essere rimossa con un singolo tocco su una qualsiasi area vuota dello schermo.
RNF-1.3	Usabilità (Temi)	L'applicazione deve adottare un'interfaccia grafica a tema scuro (<i>Dark Mode</i>) nativo per ridurre l'affaticamento visivo durante le sessioni di studio prolungate.
RNF-2.1	Regole di Validazione	Il sistema deve validare la coerenza dei voti inseriti per gli esami superati, accettando esclusivamente valori numerici compresi nel range regolamentare 18–30, prevedendo la gestione della lode.
RNF-2.2	Regole di Validazione	Il sistema deve impedire la memorizzazione di valori negativi o non numerici nei campi dedicati ai CFU e ai minuti stimati di studio.
RNF-2.3	Validazione Temporale	Il sistema deve applicare un controllo logico sulle date, impedendo la pianificazione di scadenze, esami o task in date antecedenti a quella odierna.
RNF-3.1	Affidabilità e Stato	Il sistema deve garantire la sincronizzazione istantanea dei dati tra le diverse schermate: la Dashboard deve aggiornarsi immediatamente a seguito di una mutazione effettuata nei tab specifici.
RNF-3.2	Performance	Il conteggio del timer di focus deve scorrere in modo fluido e preciso, senza subire rallentamenti o alterazioni visive a causa delle altre interazioni dell'utente sullo schermo.
RNF-3.3	Performance	Le liste di dati (corsi e scadenze) devono scorrere in modo fluido e senza ritardi percettibili anche in presenza di un numero elevato di record memorizzati.
RNF-4.1	Continuità di Servizio	L'applicazione deve garantire il pieno funzionamento di tutte le caratteristiche <i>core</i> anche in totale assenza di connettività internet (Modalità Offline).
RNF-4.2	Portabilità	Il sistema deve esibire un comportamento speculare e coerente indipendentemente dal sistema operativo mobile di destinazione (iOS e Android).

2.2 Feature avanzate scelte

Per arricchire il valore dell'applicativo e superare la natura statica dei classici planner, il gruppo ha integrato le seguenti feature avanzate:

1. **Calendario Mensile Interattivo:** Una vista calendarizzata completa che permette all'utente di avere una panoramica immediata degli appelli, delle scadenze e delle sessioni di studio programmate su base mensile, migliorando la pianificazione a lungo termine.
2. **Timer di studio integrato (Metodo Pomodoro):** Un modulo timer che permette di avviare sessioni di focus associate a specifici task, automatizzando il tracciamento del tempo effettivamente impiegato.
3. **Assistente di Studio Predittivo:** Una logica che, in base agli esami imminenti, suggerisce dinamicamente all'utente le attività prioritarie da svolgere.

2.3 Funzionalità considerate ma non implementate

In fase di design iniziale, si è valutata l'integrazione nativa con le API dei sistemi universitari (es. Esse3) per il recupero automatico del libretto e degli appelli. Tale funzionalità è stata scartata per mantenere l'applicazione agnostica e compatibile con qualsiasi ateneo, in linea con l'assenza del vincolo di utilizzo di API remote. Si è inoltre esclusa la funzionalità di login e sincronizzazione cloud per focalizzare le risorse sull'esperienza utente locale.

2.4 Limitazioni note

La limitazione tecnica principale risiede nella persistenza dei dati. In linea con i vincoli di progetto che non richiedevano lo sviluppo di un backend cloud, l'applicazione utilizza esclusivamente storage locale. Di conseguenza, i dati sono vincolati al dispositivo fisico dell'utente, non è presente una sincronizzazione cross-device e la disinstallazione dell'app comporta la cancellazione dello storico accademico.

3 Progettazione dell'app

3.1 Struttura generale dell'app

L'architettura di *StudyPlanner Pro* è stata concepita seguendo un approccio modulare e incentrato sull'utente (*User-Centered Design*). Al fine di garantire un'esperienza d'uso fluida e intuitiva, l'interfaccia è stata suddivisa in macro-aree funzionali indipendenti, ma profondamente interconnesse a livello di logica applicativa. Il sistema si basa su una *Bottom Tab Navigation* (barra di navigazione inferiore) che funge da scheletro principale dell'applicazione, permettendo allo studente di passare istantaneamente da un contesto all'altro (es. dalla pianificazione dei task al monitoraggio dei voti) senza perdere lo stato corrente della sessione.

3.2 Schermate principali

Le interfacce utente sono state raggruppate in sei schermate principali (Tab), ognuna dedicata a soddisfare specifici requisiti funzionali:

- **Dashboard (Home):** L'hub centrale dell'app. Offre un colpo d'occhio immediato sulla carriera universitaria tramite widget dinamici che mostrano gli esami imminenti, i task in scadenza e le metriche di base.
- **Corsi:** L'area dedicata al tracciamento accademico. Permette di visualizzare e filtrare i corsi, verificarne lo stato e consultare lo storico.
- **Esami:** Schermata dedicata alla pianificazione e gestione delle scadenze. Consente di monitorare gli appelli "Da Sostenere" e lo storico degli esami "Sostenuti", gestendo la logica di business relativa alle date e alle tipologie di prova.
- **Agenda e Task:** Il centro nevralgico della produttività quotidiana. Integra un calendario a scorrimento orizzontale e una To-Do List interattiva per gestire le attività di studio.
- **Timer Pomodoro:** Una schermata dedicata al Deep Work, con controlli minimalisti per scandire le sessioni di studio e associarle a specifici corsi.
- **Statistiche:** Il cruscotto analitico dell'app. Attraverso grafici a torta e barre di progresso, espone visivamente la distribuzione del tempo di studio e le proiezioni della media ponderata.

3.3 Flusso di navigazione

La navigazione tra le diverse aree funzionali dell'applicazione è orchestrata centralmente nel file `App.tsx` attraverso una struttura tabulare nativa (*Bottom Tab Navigation*). Questa scelta architetturale garantisce che l'utente non si trovi mai a più di un singolo tocco di distanza dalle funzionalità *core* dell'applicativo, azzerando i tempi di caricamento e mantenendo persistente lo stato visivo di ciascuna vista durante gli spostamenti orizzontali.

Il flusso di navigazione profonda adotta invece un pattern ibrido basato su una struttura a pila (*Stack Navigation*) gestita tramite Modali di overlay. Azioni complesse come l'aggiunta di un corso o la modifica di un esame non sradicano lo studente dalla schermata corrente, ma aprono un form in sovrimpressione. Al completamento dell'azione, il modale viene smontato dal thread grafico e lo stato della pagina sottostante si aggiorna istantaneamente in tempo reale.

3.4 Organizzazione dei dati

Sebbene l'applicazione non si appoggi a un database relazionale remoto, i dati sono stati strutturati localmente simulando un'architettura relazionale tramite l'uso rigoroso delle interfacce **TypeScript**. Lo stato globale è centralizzato tramite la **Context API** di React, fungendo da "Singola Fonte di Verità" (*Single Source of Truth*) per tutta l'applicazione.

Le entità principali del dominio applicativo sono quattro:

- **Corso:** L'entità padre. Contiene le informazioni anagrafiche della materia (nome, CFU, docente) e il suo stato (es. *Superato*, *In Corso*).
- **Task:** Entità figlia, legata univocamente a un **Corso** tramite un ID di riferimento. Contiene le informazioni operative (titolo, stato di completamento, data di scadenza).
- **Esame:** Entità dedicata al tracciamento dell'appello e alla verbalizzazione del voto, strettamente accoppiata a un **Corso**.
- **Sessione di Studio:** Log generato automaticamente al termine di un ciclo del Timer Pomodoro, contenente la durata e il riferimento al task/corso studiato per alimentare le statistiche.

Tutte le entità vengono serializzate in array di oggetti JSON e iniettate in **AsyncStorage** a ogni mutazione di stato, garantendo la persistenza.

3.5 Wireframe

Per guidare lo sviluppo dell'interfaccia utente (UI) ed esplorare l'esperienza d'uso (UX) prima della stesura del codice, è stata realizzata una progettazione strutturale preliminare tramite wireframe a bassa fedeltà (*low-fidelity*). Questo approccio ha permesso di definire la gerarchia delle informazioni, la disposizione degli elementi e i flussi di navigazione, concentrando l'attenzione sulle funzionalità piuttosto che sull'estetica.

Le scelte cromatiche definitive, orientate verso toni scuri per ridurre l'affaticamento visivo durante le sessioni di studio e l'uso di colori ad alto contrasto per le *Call to Action* (CTA) e le allerte di scadenza, sono state implementate direttamente in fase di sviluppo.

Nelle figure seguenti sono riportati i wireframe delle sei schermate principali dell'applicazione.



Figura 1: Wireframe strutturali: Dashboard (Home), Corsi ed Esami.



Figura 2: Wireframe strutturali: Agenda, Timer Pomodoro e Statistiche.

4 Scelte tecnologiche

4.1 Framework utilizzato e SDK

Per lo sviluppo dell'applicazione si è scelto di adottare **React Native**, avvalendosi del framework **Expo (SDK 54)** e del linguaggio **TypeScript**. L'architettura cross-platform ha permesso di generare un unico codice sorgente valido sia per ecosistemi Android che iOS, beneficiando al contempo della sicurezza e della robustezza garantite dalla tipizzazione statica. La scelta dell'SDK specifico e di Expo è stata guidata dalla necessità di garantire la massima compatibilità e stabilità con l'ambiente di testing **Expo Go**, riducendo i tempi di compilazione e permettendo un testing immediato in ambiente di emulazione o direttamente su dispositivo fisico tramite QR Code.

4.2 Librerie principali e Architettura UI

L'ecosistema dell'applicazione è stato volutamente mantenuto leggero, evitando l'importazione di pesanti UI Kit pre-confezionati a favore di componenti custom nativi per il totale controllo sul design e sulle performance. Le librerie fondamentali includono:

- **Expo Router:** Scelta architettonica di rilievo per la gestione della navigazione. Si è preferito adottare questo sistema di routing basato su file (*file-based routing*) per una gestione più pulita e scalabile delle schermate.
- **@react-native-async-storage/async-storage:** Utilizzata per l'interazione asincrona con il file system del dispositivo e la persistenza locale.
- **@expo/vector-icons:** Impiegata per garantire una libreria iconografica vettoriale coerente e scalabile su tutte le risoluzioni.
- **react-native-svg:** Impiegata per la renderizzazione nativa di grafiche vettoriali bidimensionali. Questa libreria è stata fondamentale per lo sviluppo custom del modulo Statistiche, consentendo la creazione di grafici a torta e barre di progresso fluidi, leggeri e perfettamente scalabili su qualsiasi risoluzione senza perdita di qualità.

4.3 Gestione avanzata dei Modali e della Tastiera Nativa

Nelle fasi di inserimento dati all'interno dei Modali di overlay (come la creazione di un nuovo task nell'Agenda), la comparsa improvvisa della tastiera di sistema rischia di coprire i campi di testo (**TextInput**). Per risolvere radicalmente questa problematica di UX (*User Experience*), sono stati combinati pattern nativi fondamentali del mondo mobile:

- **KeyboardAvoidingView basato sulla Piattaforma:** Il componente calcola l'altezza della tastiera e solleva i contenuti del form in maniera dinamica. Tramite l'oggetto **Platform**, il comportamento è differenziato a seconda del sistema operativo del dispositivo (**behavior="padding"** per iOS e **behavior="height"** per Android), garantendo la massima stabilità visiva su entrambi gli ecosistemi.

- **TouchableWithoutFeedback e Keyboard.dismiss():** Per massimizzare la naturalezza dell'interazione, l'intero modulo è avvolto da un'area cliccabile invisibile che intercetta i tocchi al di fuori della tastiera. Richiamando esplicitamente il metodo `Keyboard.dismiss()`, l'utente può nascondere istantaneamente la tastiera toccando un'area vuota.

4.4 Ottimizzazione del Rendering: Approccio Ibrido FlatList vs map()

Per la renderizzazione dei componenti grafici a schermo, si è adottato un approccio ibrido volto a bilanciare le prestazioni computazionali e la fluidità dell'interfaccia utente, prevenendo i conflitti nativi di scorrimento touch-screen.

Per le strutture dati principali, soggette a crescita dinamica e potenzialmente illimitata, è stato implementato il componente nativo **FlatList**. Questo previene il sovraccarico della memoria RAM attraverso la virtualizzazione dei nodi e il *lazy loading*, mantenendo un frame rate stabile a 60 FPS. Nello specifico, la **FlatList** è stata isolata per:

- **Elenco dei Corsi:** Gestione della lista principale degli insegnamenti in tempo reale.
- **Struttura della Dashboard:** Utilizzata come contenitore verticale primario della homepage, agganciando in coda tramite la prop `ListHeaderComponent` i widget analitici e i feed dinamici delle scadenze.

Al contrario, per la renderizzazione di elementi statici, immutabili o sotto-liste locali ad alto vincolo, si è preferito utilizzare il mappaggio JavaScript classico tramite la funzione `.map()`. Questo ha permesso di evitare l'errore strutturale di annidamento di liste verticali (*Nested VirtualizedLists Warning*), il quale disabilita l'ottimizzazione nativa. Il metodo `.map()` è stato adottato in combinazione con uno **ScrollView** orizzontale o all'interno di form per:

- **Filtri e Chip di selezione:** Pulsanti di navigazione rapida orizzontale (es. "In Corso", "Da Iniziare").
- **Task e Sessioni nei Modali:** Visualizzazione delle sotto-attività all'interno dei form di overlay, dove una lista virtualizzata avrebbe spezzato l'interazione del touch.
- **Elementi delle Statistiche:** Generazione delle legende dei grafici e delle barre di progresso annidate.

In sintesi, si sono destinate le **FlatList** esclusivamente ai feed di dati massivi verticali di primo livello, affidando al `.map()` la gestione controllata dei sotto-componenti e dei layout complessi.

4.5 Modalità di gestione della persistenza

In aderenza dei vincoli progettuali, la persistenza dei dati è gestita interamente in locale tramite **AsyncStorage**. Tutti i dati (corsi, appelli, task e metriche di studio) vengono serializzati in formato JSON al momento del salvataggio e deserializzati durante il caricamento in memoria. Questa soluzione chiave-valore si è rivelata performante e perfettamente integrabile con i flussi di stato dell'applicazione.

5 Implementazione

5.1 Organizzazione del codice e Alberatura del Progetto

L'architettura del codice sorgente di *StudyPlanner Pro* è stata strutturata seguendo i principi di modularità, manutenibilità e netta separazione delle responsabilità (*Separation of Concerns*). Sfruttando l'ecosistema Expo, il progetto adotta un'organizzazione dei file che centralizza la logica di business e rende i componenti dell'interfaccia utente atomici e riutilizzabili.

Di seguito viene mostrata l'alberatura strutturale della directory principale del progetto, utile a comprendere la distribuzione degli asset e dei file sorgente:

```
progettoMobile/
|-- app/                                # Entry-point (Expo Router)
|   |-- _layout.tsx                     # Layout e configurazione navigazione
|   \-- index.tsx                       # Punto di avvio iniziale
|-- package.json                        # Configurazione e dipendenze
|-- tsconfig.json                      # Configurazione compilatore TypeScript
\-- src/                                # Cartella radice della logica
    |-- components/                     # Componenti grafici riutilizzabili
    |   |-- CourseCard.tsx              # Rendering del singolo corso
    |   \-- SharedModals.tsx            # Modali per l'input dati
    |-- context/                         # Layer di gestione dello stato globale
    |   \-- StudyContext.tsx            # React Context API
    |-- constants/                      # Dati statici e utility temporali
    |   \-- initialData.ts              # Mock data e funzioni date ISO
    |-- types/                          # Tipizzazione del dominio
    |   \-- study.ts                    # Interfacce (Corso, Task, Esame)
    \-- screens/                        # Schermate principali (Navigazione)
        |-- DashboardTab.tsx            # Hub centrale con widget
        |-- CorsiTab.tsx                # Gestione insegnamenti
        |-- EsamiTab.tsx                # Pianificazione appelli
        |-- AgendaTab.tsx               # To-Do List e calendario
        |-- TimerTab.tsx                # Timer pomodoro
        \-- StatsTab.tsx                # Elaborazione analitica SVG
```

La cartella radice `src` isola il core dell'applicativo, disaccoppiando la logica di dominio dalla configurazione del framework. Sfruttando l'architettura *file-based routing* di Expo, la cartella `app` funge da punto d'ingresso primario. Nello specifico, il file `_layout.tsx` si occupa di montare il `StudyProvider` globale e di inizializzare la barra di navigazione inferiore (*Bottom Tab Navigation*), demandando il disegno delle singole viste ai file contenuti nella cartella `src/screens`. Il file `index.tsx`, invece, agisce come schermata di avvio e gestore dei reindirizzamenti iniziali verso la navigazione principale.

5.2 Componenti e Widget Principali

L'interfaccia utente è stata interamente scompattata in componenti personalizzati, classificabili secondo la metodologia del *Component-Driven Development* in due macro-categorie: i componenti di visualizzazione atomica e i moduli di immissione dati strutturati.

I tre elementi cardine che orchestrano l'esperienza utente e la stabilità del sistema sono:

1. **StudyContext.tsx (Il Motor dei Dati):** Non si tratta di un componente visivo, ma è il cuore logico dell'app. Sfruttando le React Context API, questo file centralizza gli array JSON di corsi, task ed esami. Espone globalmente i metodi di mutazione dello stato (funzioni asincrone di inserimento, modifica ed eliminazione) e si occupa di serializzare i dati su memoria persistente locale a ogni variazione, garantendo che un aggiornamento effettuato in una qualsiasi schermata si rifletta istantaneamente in tutte le altre.
2. **SharedModals.tsx — BaseFormModal (Layout Protetto):** È un componente strutturale custom progettato per standardizzare l'interfaccia grafica dei popup di inserimento dati (usato per registrare sessioni di studio, aggiungere task e creare corsi). Incapsula al suo interno i *Core Components* di React Native dedicati alla gestione della tastiera nativa (`KeyboardAvoidingView` e `TouchableWithoutFeedback`). Questo componente intercetta dinamicamente l'altezza della tastiera dello smartphone a seconda del sistema operativo (iOS o Android), traslando il form verso l'alto ed evitando l'occultamento dei campi di input. Consente inoltre la chiusura immediata della tastiera tramite il metodo `Keyboard.dismiss()` al tocco dello sfondo esterno.
3. **CourseCard.tsx (Riuso della Presentazione):** Un componente grafico puro (*Presentational Component*) che riceve come proprietà (*props*) l'oggetto `Course` tipizzato. Si occupa di calcolare visivamente il progresso dello studio, formattare i CFU e mostrare il badge colorato corrispondente allo stato attuale (es. blu per "In Corso", grigio per "Da Iniziare", verde per "Superato"). Viene renderizzato in modo massivo all'interno delle liste primarie del sistema.

5.3 Gestione dello Stato Globale e Persistenza dei Dati

La sincronizzazione dei dati in tempo reale tra le diverse componenti dello *StudyPlanner* è interamente affidata alla combinazione nativa di React Context API ed hook d'effetto (`useEffect`). Questo approccio architetturale esclude l'introduzione di complessi state-manager esterni, isolando la logica di business al di fuori del `StudyProvider`. Ogni transazione eseguita dall'utente (es. la verbalizzazione di un esito o la spunta di un task nell'agenda) aggiorna in modo deterministico lo stato in memoria RAM, innescando il re-rendering immediato dei soli componenti grafici interessati.

Al fine di garantire l'operatività del sistema in totale assenza di connettività di rete (Modalità Offline), ogni mutazione dello stato attiva una routine asincrona di prima istanza basata su `AsyncStorage`. Le entità del dominio (`Course`, `Task`, `Exam`), modellate e protette tramite interfacce TypeScript rigorose per preservare l'integrità dei tipi, vengono serializzate in stringhe JSON e archiviate nella memoria di massa del dispositivo mobile.

Al riavvio dell'applicazione, una singola operazione di lettura esegue il parsing dei JSON locali, ripristinando istantaneamente lo storico accademico dello studente.

5.4 Parti particolarmente significative o complesse

La complessità implementativa del software si è concentrata principalmente sulla gestione dei processi asincroni e sulla prevenzione di anomalie legate alla concorrenza dei dati. Nello specifico, lo sviluppo ha richiesto un'attenta progettazione ingegneristica in due aree nevralgiche:

1. **Gestione del Ciclo di Vita del Timer Pomodoro:** La stabilità di un countdown in ambiente React Native presenta criticità strutturali dovute al continuo re-rendering dei componenti grafici. L'utilizzo ingenuo di variabili di stato standard per il tracciamento dei millisecondi avrebbe causato una deriva temporale (sfasamento del tempo reale) e un sovraccarico della CPU. Per risolvere il problema, la logica del timer è stata isolata accoppiando l'hook `useEffect` (per l'avvio dell'intervallo nativo a 1000ms) con l'hook `useRef`. Quest'ultimo memorizza l'istanza del timer e il conteggio corrente per via referenziale, mantenendo il dato prescritto tra i rendering senza innescare calcoli grafici superflui e garantendo una precisione al millesimo di secondo.
2. **Ottimizzazione e Consistenza della Persistenza con AsyncStorage:** Trattandosi di una memoria di massa di tipo asincrono operante su stringhe chiave-valore, la sfida principale è stata garantire l'integrità referenziale dei dati in assenza di un database relazionale (come SQLite). La complessità è emersa nell'eseguire query di aggregazione e aggiornamenti simultanei: ad esempio, quando una sessione di studio viene completata, il sistema deve aggiornare in parallelo l'array dei corsi (incrementando il monte ore) e l'array delle sessioni storiche. Per evitare fenomeni di *Data Race* (corruzione del file JSON dovuto a scritture sovrapposte), l'interazione con `AsyncStorage` è stata strutturata tramite pattern `async/await`, bloccando l'esecuzione del thread fino al completamento della serializzazione e assicurando la consistenza atomica dei dati locali.

6 Manuale Operativo

6.1 Prerequisiti di sistema

Per poter eseguire correttamente l'applicazione in ambiente di sviluppo, è necessario disporre dei seguenti requisiti:

- **Node.js** (versione 18 o superiore) e il gestore di pacchetti **npm**.
- L'applicazione **Expo Go** installata sul proprio dispositivo fisico (iOS o Android), oppure un simulatore iOS/emulatore Android configurato sul computer.

6.2 Istruzioni per l'installazione e l'esecuzione

Di seguito vengono descritti i passaggi per avviare il progetto:

1. Clonare il repository GitHub in locale (o estrarre la cartella compressa).
2. Aprire il terminale, navigare all'interno della directory principale del progetto e lanciare il comando:
`npm install`
Questo scaricherà tutte le dipendenze dichiarate nel file `package.json`.
3. Al termine dell'installazione, avviare il server di sviluppo Expo tramite il comando:
`npm start` oppure `npm run start`
4. A questo punto verrà generato un QR Code nel terminale. È sufficiente inquadrarlo con la fotocamera (su iOS) o tramite l'app Expo Go (su Android) per compilare ed eseguire l'applicazione sul dispositivo fisico.

7 Metodologia di Lavoro (Extra)

7.1 Organizzazione del team e versionamento

Per lo sviluppo del progetto, il team (composto da 5 membri) ha adottato una metodologia agile, suddividendo il lavoro in micro-task assegnati individualmente. L'intero ciclo di vita del codice è stato gestito tramite **Git** e ospitato su **GitHub**.

Per evitare conflitti durante la scrittura del codice, si è adottato un flusso di lavoro basato su *branching*: ogni nuova feature (es. navigazione, timer pomodoro, UI dei corsi) è stata sviluppata su un branch separato e, solo dopo un'attenta revisione e test di funzionamento, unita al branch principale (**main**) tramite meccanismi di *Merge*.

8 Conclusioni e Sviluppi Futuri (Extra)

8.1 Considerazioni finali

Lo sviluppo di *StudyPlanner Pro* ha permesso al gruppo di consolidare le competenze legate allo sviluppo mobile cross-platform, affrontando sfide concrete relative alla gestione dello stato, del routing e della persistenza locale. L'app risultante è uno strumento coerente, stabile e pienamente rispondente ai requisiti accademici.

8.2 Roadmap per la Versione 2.0

Se l'applicazione dovesse essere pubblicata sugli store digitali in futuro, la roadmap di sviluppo per una "Versione 2.0" prevederebbe:

- **Integrazione di un Backend Cloud:** Migrazione dei dati da *AsyncStorage* a una piattaforma come *Firebase* o *Supabase*, per garantire il login utente e la sincronizzazione in tempo realtime cross-device.
- **Notifiche Push Native:** Implementazione di un sistema di notifiche (tramite *Expo Notifications*) per avvisare l'utente delle scadenze imminenti anche quando l'app è in background.
- **Gamification:** Introduzione di un sistema di obiettivi sbloccabili in base alle ore di studio completate, per incentivare la produttività dello studente.